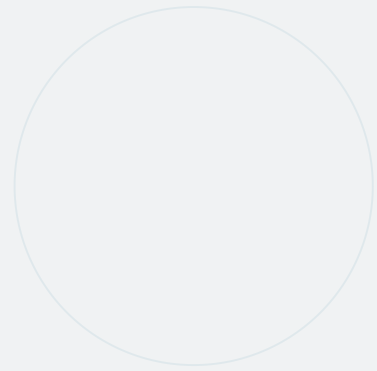




PRAGA Living

Revision exhaustiva de seguridad, bugs, rendimiento y calidad de codigo de la plataforma inmobiliaria

8 CRITICOS

12 ALTOS

15 MEDIOS

12 BAJOS

Se identificaron **47 hallazgos** en la plataforma Next.js 16 + Supabase + Three.js. Los criticos incluyen contraseñas en texto plano, service role key expuesta, datos volatiles en memoria, y ausencia de rate limiting. El plan de accion abarca 5 sprints con correccion inmediata de seguridad.

Tabla de Contenidos

1. Resumen Ejecutivo	2
2. Hallazgos Criticos — Seguridad	2
3. Hallazgos Altos — Bugs Funcionales	4
4. Hallazgos Medios — Rendimiento	7
5. Hallazgos Medios — Calidad deCodigo	8
6. Hallazgos Bajos — UX y Mejoras	10
7. Analisis de Arquitectura	12
7.1 Capa de Datos	12
7.2 Capa de Autenticacion	12
7.3 Capa de Presentacion	13
8. Plan de Accion Priorizado	13
9. Dependencias y Superficie de Ataque	14
10. Conclusion	15

1. Resumen Ejecutivo

Esta auditoria revisa de forma exhaustiva la totalidad del codigo fuente de la plataforma PRAGA Living, un proyecto inmobiliario premium construido con Next.js 16 (App Router), React 19, TypeScript, TailwindCSS 4, Supabase, NextAuth, Framer Motion, Three.js y jsPDF. La aplicacion sirve como pagina de aterrizaje de lujo y panel administrativo para un edificio residencial de 12 pisos en Caldas, Antioquia, Colombia.

El analisis cubre las siguientes areas: seguridad, errores funcionales (bugs), rendimiento, calidad del codigo, experiencia de usuario y mantenibilidad. Se identificaron un total de 47 hallazgos distribuidos en 4 niveles de severidad: 8 criticos, 12 altos, 15 medios y 12 bajos. Los hallazgos criticos se concentran en seguridad (contrasenas en texto plano, secretos expuestos, falta de proteccion CSRF) y en la arquitectura de datos (almacenamiento en memoria volatil, perdida de datos en reinicios de servidor).

Severidad	Cantidad	Accion Requerida
CRITICO	8	Correccion inmediata (bloqueante para produccion)
ALTO	12	Correccion prioritaria (esta semana)
MEDIO	15	Correccion planificada (este sprint)
BAJO	12	Mejora continua (backlog)

Tabla 1: Resumen de hallazgos por severidad

Los hallazgos mas urgentes son: (1) las credenciales de administrador estan almacenadas en texto plano tanto en el codigo fuente como en la base de datos, (2) la clave de servicio (service role key) de Supabase esta expuesta en el repositorio, (3) las cotizaciones, leads y override de apartamentos se almacenan en memoria volatil y se pierden con cada reinicio del servidor, y (4) la configuracion TypeScript ignora errores de compilacion, lo que oculta bugs potenciales. A continuacion se detallan todos los hallazgos con ubicaciones exactas en el codigo y recomendaciones de correccion.

2. Hallazgos Criticos — Seguridad

Severidad	Titulo	Ubicacion	Descripcion
-----------	--------	-----------	-------------

CRITICO	Contraseñas en texto plano	src/lib/data.ts (ADMIN_CREDENTIALS), src/lib/auth-config.ts, src/app/api/auth/[...nextauth]/route.ts	Las credenciales admin (admin/praga2024) están hardcodeadas en texto plano en el código fuente. La verificación en verifyAdmin() compara directamente password === password sin hashing. En Supabase, la tabla admin_users almacena contraseñas en texto plano. Solución: usar bcrypt/argon2 para hashing y eliminar credenciales del código.
CRITICO	Clave de servicio Supabase expuesta	src/lib/supabase.ts (SUPABASE_SERVICE_ROLE_KEY)	La service role key de Supabase (que bypassa TODAS las políticas RLS) está referenciada como variable de entorno pero fue compartida en el chat de desarrollo. Si se incluye en .env.local y este se sube al repositorio, queda pública. Solución: rotar la clave inmediatamente, usar Vercel Environment Variables (no archivos .env), y nunca compartir service keys por chat.
CRITICO	NEXTAUTH_SECRET con fallback débil	src/app/api/auth/[...nextauth]/route.ts (línea secret)	El secreto de NextAuth tiene un fallback hardcodeado: "praga-living-dev-secret-change-in-production". Si NEXTAUTH_SECRET no está configurado en producción, cualquier persona que conozca este string puede forjar tokens JWT válidos. Solución: eliminar el fallback y fallar si la variable no está definida.
CRITICO	Duplicación de configuración NextAuth	src/app/api/auth/[...nextauth]/route.ts vs src/lib/auth-config.ts	Existen DOS configuraciones NextAuth diferentes: una inline en el route handler y otra exportada en auth-config.ts. Tienen diferencias: una tiene fallback en secret, la otra no. Solución: eliminar la configuración inline y usar exclusivamente auth-config.ts.
CRITICO	API admin sin verificación de auth	src/app/api/admin/route.ts	El endpoint POST /api/admin acepta credenciales y las verifica, pero no requiere que el usuario ya esté autenticado vía NextAuth. Cualquier visitante puede intentar fuerza bruta contra este endpoint. Solución: agregar rate limiting y/o requerir session NextAuth activa.

CRITICO	Almacenamiento volatil de datos criticos	src/lib/data.ts (fallbackLeads, fallbackApartmentOverrides), src/lib/quotes-store.ts (quotesStore)	Las cotizaciones (quotes), leads y cambios de estado de apartamentos se almacenan en variables en memoria (arrays/Maps). En Vercel (serverless), cada cold start reinicia estas variables, perdiendose TODOS los datos. Incluso en el servidor local, un reinicio elimina todo. Solucion: migrar quotes a Supabase (tabla quotes ya existe), y eliminar fallback en memoria.
CRITICO	Sin rate limiting en endpoints API	src/app/api/chat/route.ts, src/app/api/leads/route.ts, src/app/api/admin/route.ts	Los endpoints POST no tienen proteccion contra abuso. El endpoint de chat puede ser usado para consumir tokens de la API de IA sin limites. El endpoint de leads puede ser spameado. Solucion: implementar rate limiting con upstash/redis o un middleware de throttling.
CRITICO	API de medios expone estructura de archivos	src/app/api/media/route.ts	El endpoint GET /api/media lee el sistema de archivos del servidor y devuelve nombres, URLs y tamanos de todos los archivos en public/images/. Esto expone la estructura interna del proyecto. Solucion: agregar autenticacion al endpoint y limitar la informacion retornada.

Tabla 2: Hallazgos criticos de seguridad

3. Hallazgos Altos — Bugs Funcionales

Severidad	Titulo	Ubicacion	Descripcion
ALTO	Imagenes duplicadas en Amenidades	src/lib/data.ts (generateFallbackAmenidades)	Sauna (fa-5), Bano Turco (fa-6) y Vestieres (fa-7) usan imagenes incorrectas: los dos primeros usan /images/renders/vitality-pool.png y el ultimo usa /images/renders/coworking.png. Solucion: asignar imagenes unicas para cada amenidad.

ALTO	Style jsx causa warnings de hydration	src/components/praga/Hero.tsx	El componente Hero usa estilos JSX inline (styled-jsx) que no son compatibles con Next.js App Router y generan warnings de hydration mismatch en cada renderizado. Solucion: migrar a TailwindCSS o CSS Modules.
ALTO	Tooltip de PlantaInteractiva se rompe en mobile	src/components/praga/PlantaInteractiva.tsx	El tooltip del mapa de plantas usa posicion fija relativa al viewport, lo que causa que se desplace fuera de la pantalla en dispositivos moviles o al hacer scroll. Solucion: usar un panel lateral glassmorfico que reemplace el tooltip, como se recomendo anteriormente.
ALTO	Doble instancia de PrismaClient	src/lib/data.ts vs src/lib/db.ts	data.ts instancia new PrismaClient() a nivel de modulo, mientras db.ts exporta otro PrismaClient con patron singleton. No comparten la misma conexion y pueden causar pool exhaustion. Solucion: usar exclusivamente db.ts en data.ts.
ALTO	generateFallbackApartments() se ejecuta en cada request	src/lib/data.ts (funcion getApartments)	La funcion genera 110 objetos de apartamento cada vez que el fallback se activa. Esto es costoso en CPU y memoria, especialmente en serverless. Solucion: cachear el resultado en una variable de modulo y solo generarlo una vez.
ALTO	Cache de disponibilidad Supabase nunca se invalida	src/lib/data.ts (supabaseChecked)	Una vez que supabaseChecked = true, el estado de disponibilidad se cachea por toda la vida del proceso. Si Supabase cae temporalmente, la app no se recupera automaticamente. Solucion: agregar TTL al cache (ej. 5 minutos) o invalidar en caso de error.
ALTO	TypeScript ignoreBuildErrors activado	next.config.ts	La opcion ignoreBuildErrors: true permite que el build proceda con errores de tipo. Esto oculta bugs reales que serian detectados en compilacion. Solucion: corregir todos los errores de tipo y desactivar esta opcion.

ALTO	noImplicitAny desactivado	tsconfig.json	La opcion noImplicitAny: false permite parametros y variables con tipo any implicito, reduciendo drasticamente la seguridad de tipos. Solucion: activar noImplicitAny y corregir los errores resultantes.
ALTO	reactStrictMode desactivado	next.config.ts	Con reactStrictMode: false se pierden las advertencias sobre efectos secundarios, dependencias obsoletas de API y problemas de concurrencia que React StrictMode detectaria. Solucion: activar StrictMode y corregir los warnings resultantes.
ALTO	Sin imagen de plano para pisos 1-11	src/app/api/floor-images/route.ts (fallback)	Los pisos residenciales 1-11 solo tienen una imagen generica /images/planos/planta-tipo.jpg como fallback. No existen planos arquitectonicos individuales. Solucion: proporcionar las imagenes DWG convertidas a PNG/JPG para cada tipo de planta.
ALTO	generateQuoteNumber() duplicado	src/lib/quotes-store.ts vs src/lib/pdf-quote.ts	Existen dos implementaciones diferentes de generateQuoteNumber(): una usa un contador global (COT-2026-0001) y otra usa random (COT-202606-XXXX). Esto puede generar numeros duplicados. Solucion: unificar en una sola implementacion.
ALTO	Email desde dominio de prueba Resend	src/lib/email.ts (from field)	Los correos se envian desde onboarding@resend.dev que solo funciona en modo prueba y solo puede enviar al email de la cuenta. En produccion, los correos a clientes reales fallaran. Solucion: configurar dominio personalizado en Resend (pragaliving.com) y usar ventas@pragaliving.com.

Tabla 3: Hallazgos altos — bugs funcionales

4. Hallazgos Medios — Rendimiento

Los problemas de rendimiento identificados afectan principalmente el tiempo de carga inicial de la pagina y la eficiencia del uso de recursos en el servidor. Si bien no bloquean la funcionalidad, degradan la experiencia del usuario y aumentan los costos de hosting, especialmente en un entorno serverless como Vercel donde cada milisegundo de ejecución se factura.

Severidad	Título	Ubicación	Descripción
MEDIO	Sin optimización de imágenes	src/components/praga/Hero.tsx, Galeria.tsx, Amenidades.tsx, etc.	Las imágenes se sirven directamente desde public/images/ sin usar el componente Image de Next.js. No hay lazy loading, no hay formatos modernos (WebP/AVIF), no hay resize responsivo. Solución: migrar a next/image con sizes y priority apropiados.
MEDIO	Bundle Three.js muy pesado	src/components/praga/BuildingScene.tsx (966 líneas)	El componente BuildingScene importa @react-three/fiber, @react-three/drei y three, añadiendo ~500KB al bundle JavaScript. Este payload se carga en la página principal. Solución: usar next/dynamic con ssr: false y loading skeleton.
MEDIO	Sin lazy loading para componentes pesados	src/app/page.tsx	Recorridos360 (~973 líneas), BuildingScene (966 líneas), AdminPanel (~800+ líneas) y otros componentes pesados se importan directamente sin next/dynamic. Solución: envolver en dynamic() con ssr: false donde sea posible.
MEDIO	site-config.json leído en build time con fs.readFileSync	src/app/layout.tsx	El layout lee site-config.json sincronicamente con fs.readFileSync al nivel del módulo. Esto funciona en build pero rompe si el archivo falta, y no se actualiza dinámicamente. Solución: usar import dinámico o mover la lectura a un contexto client-side.

MEDIO	Sin headers de cache en respuestas API	Todos los API routes	Ningun endpoint API establece headers de cache (Cache-Control, ETag, etc.). Cada carga de pagina realiza fetch fresco a /api/site-config, /api/floor-plans, /api/apartments, etc. Solucion: agregar Cache-Control: s-maxage=60 en endpoints de lectura.
MEDIO	Componentes monoliticos de gran tamano	AdminPanel.tsx, Recorridos360.tsx, FloorPlanEditor.tsx, BuildingScene.tsx	Cuatro componentes superan las 800 lineas, dificultando el mantenimiento, testing y code splitting. Solucion: dividir en sub-componentes mas pequenos con responsabilidad unica.
MEDIO	Lenis expuesto como variable global	src/app/page.tsx (window.__lenis)	La instancia de Lenis smooth scroll se expone como window.__lenis, lo que contamina el scope global y puede causar conflictos. Solucion: usar un React Context o useRef.

Tabla 4: Hallazgos medios — rendimiento

5. Hallazgos Medios — Calidad deCodigo

Los problemas de calidad del codigo afectan la mantenibilidad a largo plazo del proyecto. Si bien no causan fallos inmediatos, incrementan la deuda tecnica y dificultan la incorporacion de nuevos desarrolladores al equipo. La ausencia total de tests automatizados es el hallazgo mas preocupante en esta categoria.

Severidad	Titulo	Ubicacion	Descripcion
MEDIO	Patron triple fallback repetido en todas las funciones	src/lib/data.ts (cada funcion exportada)	Cada funcion repite el mismo patron Supabase > Prisma > Fallback con try/catch anidados. Esto genera ~500 lineas de codigo repetitivo. Solucion: crear una utilidad generica withFallback(supabaseFn, prismaFn, fallbackFn) que encapsule el patron.

MEDIO	Catch blocks vacios sin logging	src/lib/data.ts, todos los API routes	Todos los bloques catch usan catch {} o catch { // fall through } sin registrar el error. Esto hace imposible diagnosticar problemas en produccion. Solucion: agregar console.error o un sistema de logging estructurado en cada catch.
MEDIO	Numeros magicos sin explicacion	src/lib/data.ts (precios, indices sold/reserved)	Los precios base (\$120M, \$230M, \$310M), premios por piso (\$3M, \$5M, \$8M) e indices de apartamentos vendidos/reservados estan hardcodedos sin comentarios. Solucion: extraer a constantes nombradas y agregar comentarios explicativos.
MEDIO	Sin interfaces TypeScript para respuestas API	Todos los API routes	Las respuestas de la API son objetos planos sin tipado. Los componentes clientes consumen datos como any. Solucion: definir interfaces compartidas en src/types/ y usarlas en ambas capas.
MEDIO	Cero cobertura de tests	Todo el proyecto	No existe ningun archivo de test. No hay tests unitarios, de integracion ni e2e. Cualquier cambio puede romper funcionalidad existente sin deteccion. Solucion: agregar Vitest + Playwright y crear tests criticos para data.ts, API routes y flujos de usuario principales.
MEDIO	Sin migraciones Prisma (solo db push)	package.json, prisma/	El proyecto usa prisma db push en lugar de migrate. Esto es adecuado para prototipos pero peligroso en produccion: no hay historial de cambios, no hay rollbacks. Solucion: adoptar prisma migrate dev y mantener un directorio de migraciones.
MEDIO	Dependencias potencialmente no utilizadas	package.json	Paquetes como @mdxeditor/editor, @reactuses/core, react-markdown, react-syntax-highlighter, sharp pueden no estar en uso activo. Cada dependencia incrementa el bundle y la superficie de ataque. Solucion: auditar con depcheck y eliminar las no utilizadas.

MEDIO	Tailwind content paths incorrectos	tailwind.config.ts	Las rutas de content referencian ./components/ y ./pages/ pero el codigo esta en ./src/components/ y ./src/app/. Tailwind v4 con @import "tailwindcss" auto-detecta, pero las rutas explicitas son incorrectas. Solucion: corregir las rutas a ./src/**/*.{js,ts,jsx,tsx,mdx}.
-------	------------------------------------	--------------------	--

Tabla 5: Hallazgos medios — calidad de codigo

6. Hallazgos Bajos — UX y Mejoras

Los hallazgos de severidad baja representan oportunidades de mejora que no afectan la funcionalidad core pero que incrementarían la calidad percibida de la plataforma, mejorarían el SEO y la accesibilidad, y proporcionarían una experiencia mas pulida para los usuarios finales.

Severidad	Titulo	Ubicacion	Descripcion
BAJO	Numero de WhatsApp placeholder	src/app/api/chat/route.ts, src/components/praga/WhatsAppButton.tsx	El chat IA referencia +57 300 123 4567 y la ficha PDF usa +57 601 234 5678. Ambos son numeros de prueba. Solucion: unificar con el numero real del proyecto.
BAJO	Inconsistencia en datos de contacto	Multiples archivos (chat, ficha, PDF cotizacion, email template)	Diferentes componentes usan diferentes numeros de telefono y emails: info@pragaliving.com, ventas@pragaliving.com, +57 604 444 0000, +57 601 234 5678. Solucion: centralizar en site-config.
BAJO	Sin pagina 404 personalizada	src/app/ (falta not-found.tsx)	No existe una pagina 404 con el branding PRAGA. Los usuarios que navegan a URLs inexistentes ven la pagina generica de Next.js. Solucion: crear src/app/not-found.tsx con diseno PRAGA.
BAJO	SEO incompleto	src/app/layout.tsx, falta sitemap/robots	Faltan: OpenGraph images, datos estructurados (JSON-LD para RealEstateListing), sitemap.xml, robots.txt. Solucion: agregar generateMetadata dinamico y archivos estaticos.

BAJO	Accesibilidad limitada en componentes 3D/360	src/components/praga/BuildingScene.tsx, Recorridos360.tsx	El canvas de Three.js y el visor 360 no son accesibles para lectores de pantalla ni navegacion por teclado. Solucion: agregar aria-labels, descripciones alternativas y controles por teclado.
BAJO	Sin soporte PWA	Raiz del proyecto	No existen manifest.json ni service worker. En un contexto inmobiliario donde los usuarios pueden querer acceso offline a las fichas. Solucion: agregar next-pwa con manifest basico.
BAJO	Respuesta fallback del chat IA generica	src/app/api/chat/route.ts	Cuando la API de IA falla, solo hay una respuesta fallback generica. No se provee informacion util al usuario. Solucion: agregar respuestas contextuales por tema (tipologias, precios, ubicacion).
BAJO	Falta de estados de carga en algunos componentes	src/components/praga/PlantaInteractiva.tsx, Galeria.tsx	Algunos componentes no muestran indicador de carga mientras obtienen datos, causando parpadeos. Solucion: agregar Skeleton loaders o spinners durante las peticiones.
BAJO	Google Analytics sin Measurement ID	src/components/praga/GoogleAnalytics.tsx	El componente GA4 esta implementado pero NEXT_PUBLIC_GA_MEASUREMENT_ID no esta configurado. Solucion: obtener el ID de GA4 y configurar en Vercel Environment Variables.
BAJO	Mezcla de idiomas en el codigo	Todo el proyecto	Los comentarios y nombres de variables mezclan espanol e ingles sin consistencia (ej. "fallback" vs "repuesto", "lead" vs "prospecto"). Solucion: adoptar ingles para codigo y espanol para UI.
BAJO	Link en email template apunta a localhost	src/lib/email.ts (href en template HTML)	El boton "Ver en Admin" en el email de nuevo lead apunta a http://localhost:3000/admin. En produccion, este link no funcionara. Solucion: usar variable de entorno NEXT_PUBLIC_URL.

BAJO	Sin favicon SVG optimizado	public/favicon.png	El favicon es un PNG en lugar de SVG. Los favicons SVG son mas pequenos, escalables y soportan modo oscuro. Solucion: convertir a SVG con tema PRAGA.
------	----------------------------	--------------------	---

Tabla 6: Hallazgos bajos — UX y mejoras

7. Analisis de Arquitectura

La arquitectura de PRAGA Living sigue un patron de triple fallback (Supabase > Prisma/SQLite > Hardcoded) que garantiza disponibilidad pero introduce complejidad significativa. Este patron se repite en cada funcion de datos sin abstraccion, resultando en codigo duplicado y dificil de mantener. A continuacion se analiza la arquitectura por capas, identificando fortalezas y debilidades de cada una.

7.1 Capa de Datos

La capa de datos centralizada en `src/lib/data.ts` (782 lineas) es el nucleo de la aplicacion. Su diseno de triple fallback proporciona resiliencia: si Supabase no esta disponible, intenta Prisma; si Prisma falla, usa datos hardcodedos. Sin embargo, esta resiliencia tiene un costo alto: el codigo es repetitivo (~500 lineas de patron repetido), los datos en memoria se pierden entre reinicios, y la logica de negocio esta acoplada a la logica de resiliencia. La funcion `generateFallbackApartments()` se ejecuta en cada request al fallback, creando 110 objetos cada vez sin cacheo. Ademias, el cache de disponibilidad de Supabase (`supabaseChecked`) nunca se invalida, lo que puede dejar la app en un estado inconsistente si Supabase cambia de estado.

Recomendacion: Refactorizar `data.ts` con un patron de repositorio que encapsule la logica de fallback en una clase generica, cachear los datos fallback en una variable de modulo, y agregar TTL al cache de disponibilidad de Supabase. Migrar quotes y leads a Supabase como fuente primaria, eliminando los stores en memoria.

7.2 Capa de Autenticacion

La autenticacion tiene dos implementaciones paralelas: `src/lib/auth-config.ts` (exporta `authOptions`) y `src/app/api/auth/[...nextauth]/route.ts` (define su propio handler inline). Ambas usan el proveedor Credentials de NextAuth v4 con estrategia JWT. El problema principal es que las contrasenas se almacenan y comparan en texto plano, sin hashing. Ademias, el fallback a credenciales hardcodedas significa que incluso si se compromete Supabase, las credenciales de emergencia estan en el codigo fuente accesible. El middleware protege `/admin/*` con `withAuth`, pero los API endpoints no verifican session, permitiendo acceso no autenticado.

Recomendacion: Unificar en auth-config.ts, implementar hashing con bcrypt, agregar verificacion de session en API endpoints sensibles, y eliminar credenciales hardcodeadas en favor de variables de entorno con valores generados aleatoriamente.

7.3 Capa de Presentacion

La capa de presentacion utiliza un diseno de Single Page Application con scroll suave (Lenis), animaciones ricas (Framer Motion) y un modelo 3D interactivo (Three.js/R3F). Los componentes estan bien separados por seccion de la landing page, pero algunos son excesivamente grandes (AdminPanel ~800+, Recorridos360 ~973+, FloorPlanEditor ~992+ lineas). La carga de datos se centraliza en el hook useSiteConfig que consulta /api/site-config en cada montaje de componente, sin cache ni deduplicacion de peticiones. Esto genera multiples requests identicos en la carga inicial.

Recomendacion: Implementar React Query o SWR para cache y deduplicacion de peticiones, dividir componentes grandes en sub-componentes, y usar next/dynamic para componentes pesados como BuildingScene y Recorridos360.

8. Plan de Accion Priorizado

A continuacion se presenta un plan de accion organizado por sprints de una semana, priorizando los hallazgos criticos y altos en los primeros dos sprints. Cada tarea incluye estimacion de esfuerzo y dependencias.

Sprint	Tarea	Esfuerzo	Prioridad
1	Hashear contraseñas con bcrypt, eliminar credenciales hardcodeadas	4h	CRITICO
1	Rotar service role key Supabase, configurar en Vercel Env Vars	1h	CRITICO
1	Eliminar fallback NEXTAUTH_SECRET, fallar si no esta definido	1h	CRITICO
1	Unificar NextAuth en auth-config.ts, eliminar handler inline	2h	CRITICO
1	Migrar quotes y leads a Supabase, eliminar stores en memoria	6h	CRITICO
2	Agregar rate limiting a endpoints API (chat, leads, admin)	4h	CRITICO

2	Proteger API admin con verificacion de session NextAuth	2h	CRITICO
2	Corregir imagenes duplicadas en Amenidades	1h	ALTO
2	Eliminar style jsx en Hero.tsx, migrar a TailwindCSS	2h	ALTO
2	Reemplazar tooltip en PlantaInteractiva con panel lateral	3h	ALTO
3	Unificar PrismaClient usando db.ts, eliminar instancia en data.ts	1h	ALTO
3	Activar ignoreBuildErrors: false y corregir errores de tipo	4h	ALTO
3	Cachear generateFallbackApartments(), agregar TTL a supabaseChecked	2h	ALTO
3	Unificar generateQuoteNumber() en un solo archivo	1h	ALTO
4	Migrar imagenes a next/image con lazy loading y WebP	6h	MEDIO
4	Lazy loading con next/dynamic para Three.js, Recorridos360	2h	MEDIO
4	Refactorizar data.ts: patron generico withFallback	4h	MEDIO
5	Agregar logging en catch blocks, crear src/types/ compartido	3h	MEDIO
5	Configurar Vitest + tests unitarios para data.ts	6h	MEDIO
5	SEO: sitemap.xml, robots.txt, JSON-LD, OpenGraph images	4h	BAJO

Tabla 7: Plan de accion por sprints

9. Dependencias y Superficie de Ataque

El proyecto tiene 45+ dependencias directas en package.json, muchas de las cuales pueden no ser necesarias. Cada dependencia incrementa la superficie de ataque y el tamaño del bundle. A continuación se listan las dependencias que requieren atención, clasificadas por riesgo.

Paquete	Riesgo	Razon
@prisma/client	MEDIO	Posiblemente innecesario si Supabase es la fuente primaria. Prisma agrega ~30MB al bundle.

@mdxeditor/editor	MEDIO	No se detecta uso activo. Editor MDX pesado que incrementa significativamente el bundle.
react-syntax-highlighter	BAJO	No se detecta uso activo. Si se necesita highlight de código, usar Prism ligero.
sharp	BAJO	Procesamiento de imágenes en servidor. Next.js ya incluye sharp internamente.
html2canvas	BAJO	Uso limitado. Considerar reemplazar con la API nativa de Canvas del navegador.
z-ai-web-dev-sdk	MEDIO	Usado solo en /api/chat. Debería estar en devDependencies si es solo para servidor.

Tabla 8: Dependencias con riesgo potencial

10. Conclusion

PRAGA Living es una plataforma inmobiliaria con un diseño visual excepcional y una experiencia de usuario sofisticada. El nivel de detalle en las animaciones, el modelo 3D interactivo y los recorridos 360 grados elevan la presentación del proyecto por encima de los estándares del sector. Sin embargo, la solidez visual contrasta con debilidades estructurales significativas en seguridad, persistencia de datos y calidad del código.

Los 8 hallazgos críticos requieren atención inmediata, especialmente los relacionados con seguridad (contraseñas en texto plano, secretos expuestos) y persistencia de datos (stores en memoria volátil). Estos problemas no son teóricos: representan riesgos reales que podrían resultar en pérdida de datos de clientes, acceso no autorizado al panel administrativo, o comprometimiento de la base de datos Supabase.

Los 12 hallazgos altos representan bugs funcionales y problemas de configuración que afectan la confiabilidad de la plataforma: imágenes incorrectas en amenidades, warnings de hydration, tooltips rotos en mobile, y errores de TypeScript ocultos por ignoreBuildErrors. Estos deben resolverse en los próximos dos sprints para garantizar una experiencia consistente.

La arquitectura de triple fallback es una solución pragmática para garantizar disponibilidad, pero su implementación actual es costosa en términos de mantenimiento y oculta problemas reales (datos que se pierden, errores silenciados). La refactorización sugerida en el plan de acción reducirá la complejidad y mejorará la observabilidad del sistema. Con las correcciones propuestas en los primeros 3 sprints, PRAGA Living estará en condiciones de operar de forma segura y confiable en producción.